



Technical Doc

👤 Created by	 Judds Art
🕒 Created time	@September 19, 2025 12:21 AM
👤 Last edited by	 Judds Art
🕒 Last updated time	@December 10, 2025 2:00 AM

Rooftop Garden

Author	Date	Version	Comments
Ioana Raileanu	3.9.2025	0.0.1	Document created
Ioana Raileanu	10.09.2025	0.1.0	Updated document
Ioana Raileanu	11.09.2025	1.0.0	Finished first draft
Ioana Raileanu	16.11.2025	1.1.0	Updates
Ioana Raileanu	08.12.2025	2.0.0	Finished tech doc

This document is best viewed on Notion [here](#).

Table of contents

[Rooftop Garden](#)

[Table of contents](#)

[Game / Software overview](#)

[Core features](#)

[Technical goals](#)

[Platforms and system requirements](#)

[Tools and third-party libraries](#)

[Coding standards](#)

[C++ Code](#)

[Naming Convention](#)

[Blueprint Code](#)

[Naming Convention](#)
[Folder structure](#)
[Gitlab base folder structure](#)
[Unreal project folder structure](#)
[each folder can be further sub-divided into categories](#)
[Art assets folder structure](#)
[SW Architecture](#)
[Class diagrams](#)
[Application flowchart](#)

Game / Software overview

Rooftop Garden is a first-person single player game about farming in a rooftop garden. You are trapped in a compact room, with nothing but your garden to keep you company. Your only friend is the omnivorous plant that mysteriously lives on your rooftop.

You have to grow your own produce to keep yourself alive, as well as avoid the omnivorous plant's wrath.

Core features

The main mechanic of the game is farming, and it consists of the following sub-mechanics:

- Ploughing the soil
- Planting the seeds
- Watering the plants
- Removing bugs and weeds from the plants (if necessary)
- Collecting your produce
- Quest box with needed items and rewards

The farming mechanic is supported by the day-night cycle feature, as well as by the omnivorous plant, which plays an important role in the farming process: it will sometimes give you random seeds to expand your garden.

Another mechanic branching from the farming is the hunger meter: yours, and the omnivorous plant's. You have to grow

enough food to maintain both of you.

Extra food/items should be used for quests, as they can give you progressive items.

There's a simple crafting system set in place as well, for crafting sprays like Herbicide, Pesticide or Fertilizer, or for obtaining more seeds from produce.

Technical goals

- Reliability
 - The game should not have any existing crashes/errors by delivery date
 - Error handling is consistently done from the start
- Maintainability
 - It should be really easy to build on top of the current framework
 - Component/Interfaces are preferred over casting
- Scalability
 - It's easy to add new features on top of the pre-existing systems
- Data-driven design
 - The artists/designers can easily configure new game content through simple blueprint/data table changes, without having to mess with the code
- Target framerate: 60fps
- Supported input device: gamepad or keyboard and mouse

Platforms and system requirements

Platforms: Windows 10/11 (64-bit)

System requirements:

- Minimum:
 - **OS:** Windows 10 64-bit

- **Processor:** Dual-core CPU (Intel i3-8100 / AMD Ryzen 3 1200 or equivalent)
- **Memory:** 8 GB RAM
- **Graphics:** NVIDIA GTX 1050 Ti / AMD RX 560 (4 GB VRAM)
- **Storage:** ~5-10 GB available
- **DirectX:** Version 12
- **Recommended:**
- **OS:** Windows 11 64-bit
- **Processor:** Quad-core CPU (Intel i5-9600K / AMD Ryzen 5 3600 or equivalent)
- **Memory:** 16 GB RAM
- **Graphics:** NVIDIA GTX 1660 Super / AMD RX 6600 (6 GB+ VRAM)
- **Storage:** SSD with ~10 GB free space
- **DirectX:** Version 12

Tools and third-party libraries

- Operating System: *Windows 10/11*
- Game Engine: *Unreal Engine 5.4.4*
 - Plugins: Visual Studio Tools, RiderLink, Socket Preview (self-developed)
- Programming language: *C++ & Blueprints*
- Source Control: *GitLab (hosted by Kamk/Kamit)*
- Modeling, Rigging, Animation: *Blender 4.4.1, Maya 2026.3*
 - Blender addon: *Super Batch Export*
- Development Environment: *Visual Studio 2022 17.14.4, JetBrains Rider 2025.3.0.1*
- Textures: *Substance Painter 11.0.1, Krita 5.2.9, Blender 4.4.1, Figma*
- Task Management: Jira and Trello

- Idea dump: Miro

Coding standards

C++ Code

For a more comprehensive list of the coding standards used in this project, please check Unreal's Coding Standard. The more important coding standards are listed below.

- **Raw pointers should almost never be used!** Unreal's type `TObjectPtr` is preferred (e.g. `TObjectPtr<FStaticMesh>`)
- **All public variables need to be marked as `UPROPERTY!`** Otherwise the garbage collector will create issues.
- **Globals should NOT be used!** Use the game instance or `UworldSubsystem` for something similar.
- It's not always necessary to check if a variable is valid! If the variable is supposed to always be valid, printing an error message when it's not is expected instead of just performing when the variable is valid.
- Functions that return a boolean should ask a question (e.g. `IsCollectable`)
- Comments over non-explanatory public functions should always be included, further explaining the function's utility (and parameters)

Naming Convention

Generally, the base naming convention for Unreal Engine is PascalCase. There shouldn't be any underlines in the names. Unreal's types have an additional upper-letter prefix (e.g. `FObject`):

- Template classes are prefixed by `T`
- Classes that inherit from `UObject` are prefixed by `U`
- Classes that inherit from `AActor` are prefixed by `A`
- Classes that inherit from `SWidget` are prefixed by `S`

- Classes that are abstract interfaces are prefixed by I
- Enums are prefixed by E
- Boolean variables must be prefixed by b
- Most other classes are prefixed by F

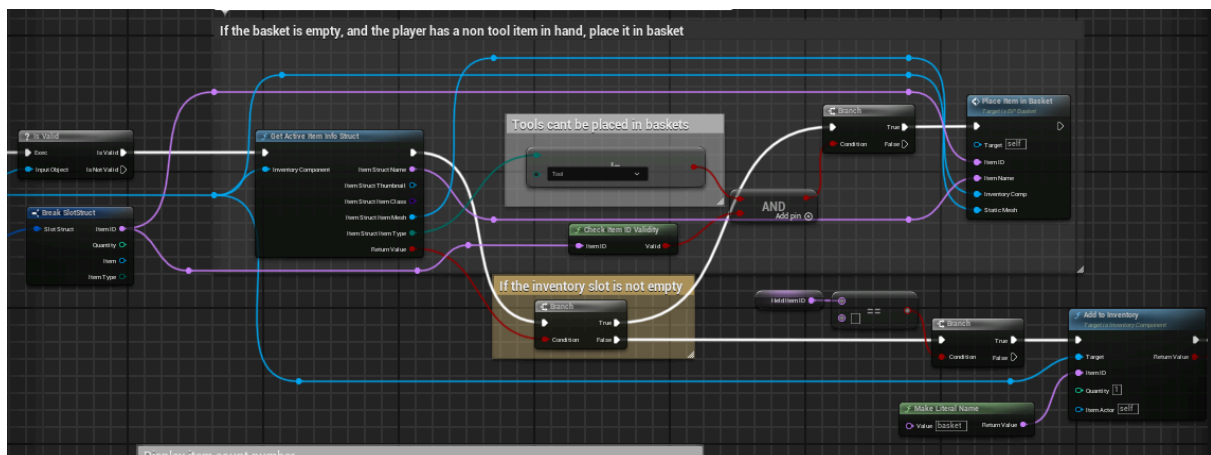
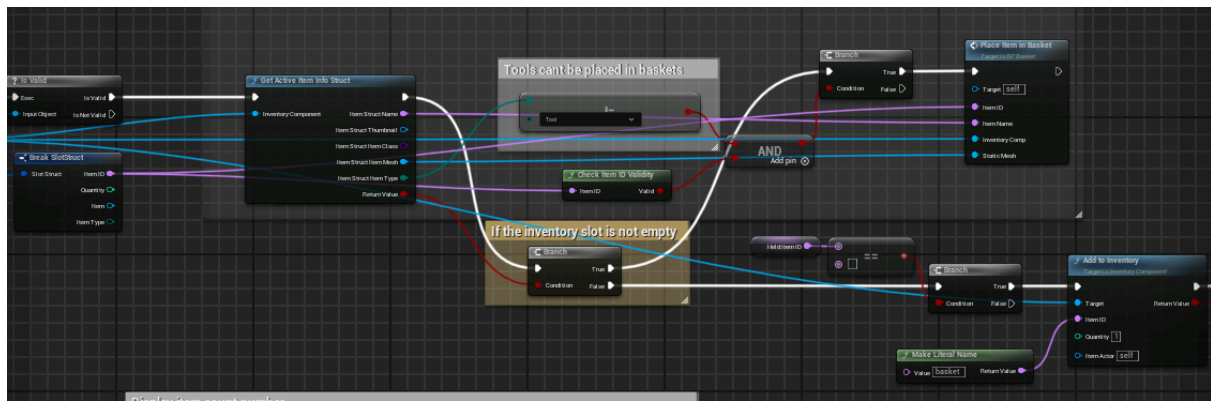
For asset naming conventions refer to the [Unreal Directive](#).

For asset naming convention inside the ArtAsset folder use the following main rules:

- Texture prefix is T_ , followed by the mesh name and the map type as a suffix "T_[AssetName]_[MapType]". Use the following as suffixes:
 - _BaseColor
 - _Normal
 - _Opacity
 - _Roughness
 - _AO
 - _Metallic
- SM_MeshName for static meshes
- SKM_MeshName for skeletal (rigged) meshes
- AS_AnimationName for animations

Blueprint Code

- Code should always be separated into events/functions when it starts getting too long and messy
- Comments over big blocks of blueprints that specify the functionality of that code are highly recommended (with the popup turned on)
- Comments over smaller blocks of code that are unclear/harder to understand are also highly encouraged
- Nodes should be nicely aligned with blueprint blocks to avoid spaghetti code that is hard to read. Example:



Naming Convention

- Pascal case is used for variables, events and functions. No underlines or special characters should be used.

Folder structure

Gitlab base folder structure

ArtAssets

Source - contains all source art assets used in the game

Prototyping - used for tests

Exports - contains all the exported assets (e.g. .fbx) in the game. Updating an exported asset is done by rewriting the original file, so it can be easily reimported in Unreal's editor by *RMB->Reimport*

Dev - contains one Unreal project: the main development folder used in the production phase

📁 **Main** - contains a stable version of the game - receives regular updates from the Dev folder

📁 **Prototype** - contains tests and experiments; will be used mostly during the prototype phase

Unreal project folder structure

each folder can be further sub-divided into categories

📁 Animations

📁 AnimBlueprints

📁 AnimMontages

📁 AnimNotifies

📁 AnimSequences

📁 BlendSpaces

📁 Audio

📁 SoundCues

📁 SoundWaves

📁 Blueprints

📁 Pawns

📁 Actors

📁 Components

📁 DataTypes

📁 Game -> for GameInstance, GameMode, HUD, etc.

📁 Particles

📁 Widgets

📁 Misc

📁 Fonts

📁 Input

📁 Actions

📁 Maps

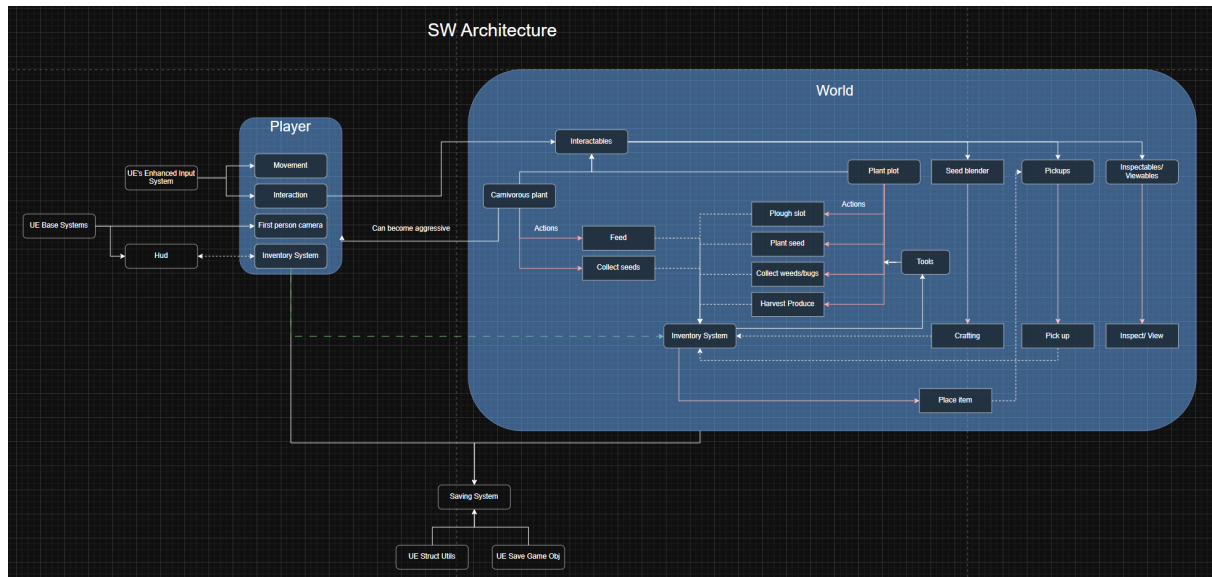
- 📁 **Debug**
- 📁 **Materials**
 - 📁 **MaterialInstances**
- 📁 **Meshes**
 - 📁 **PhysicsAssets**
 - 📁 **SkeletalMeshes**
 - 📁 **Skeleton**
 - 📁 **StaticMeshes**
- 📁 **Textures**

Art assets folder structure

The art folder should mainly resemble the Unreal project's structure, with the addition of being further subdivided into categories.

- 📁 **Concept art**
- 📁 **Prototype**
- 📁 **Meshes**
 - 📁 **Plants**
 - 📁 **PlantName** – includes all growth stages
 - 📁 **Environment**
 - 📁 **Animals**
 - 📁 **Tools**
- 📁 **Textures**
 - 📁 **Plants**
 - 📁 **PlantName**
 - 📁 **Environment**
 - 📁 **Animals**
 - 📁 **Tools**
 - 📁 **UI**

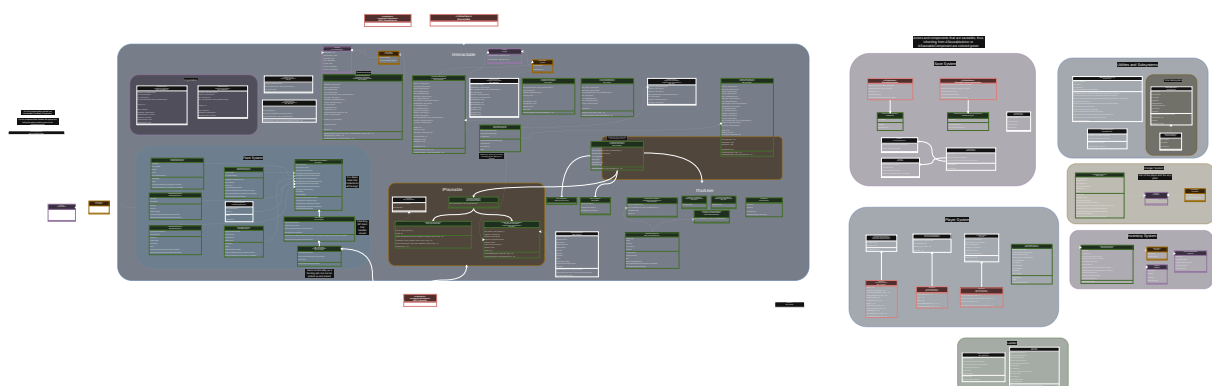
SW Architecture



The one system that connects to all interactables is the inventory system, since it involves either using something from your inventory, or collecting something into your inventory.

All interactable objects use the `Interactable` interface. All placeables use the `Placeable` interface, and all tools use the `ToolUser` interface.

Class diagrams



- The SVG file shown above can also be downloaded from the top right of the image.

- The [draw.io](#) URL can be accessed [here](#).

Application flowchart

